- This cache coherency protocol is based on a distributed collection of L1 caches that all can be both read from or written to.
  - For the purposes of this document, it seems easier to rely on having each L1 icaches just be implemented the same as an L1 dcache for simplicity's sake.
- All of the L1 caches will be hooked up to a central directory.
- If multiple L1 caches share a cache line, then we have the following situations:
  - The directory will forward any write by a CPU to any shared cache line to each L1 cache that has a copy of said shared cache line.
    - A priority arbiter will block a CPU from writing to its attached L1 dcaches while the central directory is forwarding a write to that L1 dcache.
    - Write forwarding is done in a serialized manner.
      - I was thinking of having a (new?) module that is similar to a reorder buffer, sort of, to implement the serialization.
        - This module would have, I suppose, one write port per L1 dcache (which is most likely the same number as the number of CPU cores). (L1 icaches don't have to worry about this specific thing, which should help with all three of performance in cycles, performance as in clock rate, and also area).
        - Perhaps need to use the `io.forceHost` input of each L1 cache's write port's priority arbiter to prevent the attached CPU from accessing the L1 while the central directory has a to-be-forwarded write queued? It looks like the existing implementation of `io.forceHost` was done in such a way that it could be made use of for this purpose.
  - The directory needs to avoid actually sending evicted L1 dcache lines to `io.dirHiBus` **iff** the L1 dcache line is located in another L1 dcache.
    - Also, I think I need a tiny addition to `io.dirHiBus.h2dBus.cacheInfo.payload` to include the index of the L1 dcache that is attempting to evict a cache line.
  - When an L1 cache fetches a fresh copy of a cache line upon a cache miss (i.e. via its `io.hiBus` interface) the directory will look to see whether that cache line is already in another L1 cache, and if so, the directory will forward that request to the first L1 cache it can find that has a copy of that cache line.
    - I think this means I need to add a third "request" interface to the priority arbiter in front of each L1 cache's "request" port. The priorities are as follows, with lower numbers in the list indicating a higher priority than higher numbers in the list:
      1. Writes to an L1′s shared cache lines.
      2. fetching a shared cache line (i.e. upon an L1 cache having a miss).
      3. The L1 cache's attached CPU's usual/typical requests.
    - So I think I could use a modification to my priority arbiter to make it so that as long as there is exactly one requester trying to access the arbitrated interface (`io.dev` was its name IIRC), perhaps continue allowing that host to access the arbitrated interface such that it's done in a pipelined manner. It is also possible that I could do that as long as `io.forceHost` is active I guess?
      - This kind of change would make it so that an L1 icache, at least, can still have its existing bus response throughput of one-instruction-per-cycle when it a write isn't being forwarded to said L1 icache. This actually would help with L1 dcaches as well, but I was more worried about L1 icaches because I at **least** want the L1 icache to be able to output one-instruction-per-cycle.